

CSCI 232

232 Algorithms & Data Structures

LECTURE: 14

J. L. Pach

OUTLINE

- Recap: from `struct` + functions to `class` + methods
- Class diagram (UML): `DoublyLinkedList` and its private `DoubleNode`
- Composition: who owns whom
- Constructor, destructor, `new` / `delete`
- RAII, value semantics and the Rule of Three
- Public API (`push`, `pop`, `delete`) built on private `detach` helpers

FROM C TO C++

Recap

LECTURE 08 VS LECTURE 14

Lecture 08 (C)	Lecture 14 (C++)
`struct Node`	private nested `DoubleNode`
`initNode(&a, 10)`	constructor `DoubleNode(10)`
`createNode` → `malloc`	`new DoubleNode(10)` inside `pushBack`
`destroyNode(&ptr)` → `free`	`delete` inside the class
`Node **headPtrPtr`	`headPtr` is a **member**
`detachFirst(&headPtr)` public	`detachFirst()` **private**, public `popFront(out)`

> The list no longer needs `Node **`.
>
> The pointer lives **inside the object** — methods see it through `this`.
>
> Nodes never leave the class.

CLASS DIAGRAM

and Composition

WHAT IS A CLASS DIAGRAM?

A **class diagram** (UML) shows:

- classes — name, attributes, methods,
- visibility — `-` private, `+` public,
- relationships between classes.

It is a plan **before** we write any code.

```
+---------------------+
|     ClassName       |
+---------------------+
| - attribute : type  |
+---------------------+
| + method() : type   |
+---------------------+
```

Class Diagram — Our Design



`<#>` filled diamond = **composition**.

> `DoubleNode` is private. The outside world works with `int`, never with nodes.

COMPOSITION — WHAT IT MEANS

Composition = "has-a" with **ownership**.

- `DoublyLinkedList` **owns** its `DoubleNode`s.
- Nodes are created by the list (`new`).
- Nodes are destroyed by the list (`delete`).
- A node does not live on its own inside the list.
- When the list dies — **all its nodes die with it**.

```
DoublyLinkedList  <>------  DoubleNode
    (whole)                 (part)
```

> Whole destroyed → parts destroyed.
> That is exactly the job of the **destructor**.

COMPOSITION VS AGGREGATION

	Composition `<#>--` (filled)	Aggregation `<>--` (empty)
Relation	strong "owns"	weak "uses"
Part lifetime	tied to whole	independent
Example	List and Nodes	Team and Players
Who deletes?	the whole	someone else

> Linked list = composition.
>
> If the list does not delete its nodes, you have a **memory leak**.

DOUBLENODE

The Part

DOUBLENODE: NESTED PRIVATE CLASS

```
class DoublyLinkedList
{
private: struct DoubleNode              // invisible outside the list
   {
      int         value;
      DoubleNode *prevPtr;
      DoubleNode *nextPtr;

      explicit DoubleNode(int v)
         : value(v), prevPtr(nullptr), nextPtr(nullptr) {}
   };
   // ... rest of the list ...
};
```



DOUBLENODE — CLASS

```
class DoubleNode
{
private:     int         value;
private:     DoubleNode  *prevPtr;
private:     DoubleNode  *nextPtr;
private:    friend class DoublyLinkedList;   // the owner may touch links

public: explicit DoubleNode(int value) : value(value), prevPtr(nullptr), nextPtr(nullptr) {}
public:     int getValue() const { return value; }
};
```



DOUBLENODE — NOTES

- The node is an **implementation detail**. Code outside the list cannot even write the name `DoubleNode`.
- Its fields are public **inside** the nested struct, but the struct itself is private, so only the list can reach them. No `friend` needed.
- The constructor initializes **everything**. No uninitialized node can exist.
- `explicit` blocks accidental conversion `int → DoubleNode`.

> This is composition enforced by the compiler:
> the part is hidden, and only the whole can create, link and delete it.

DOUBLYLINKEDLIST

The Whole

DoublyLinkedList: Class Skeleton

```
class DoublyLinkedList
{
private:
   struct DoubleNode { /* see previous slide */ };

   DoubleNode *headPtr;
   DoubleNode *tailPtr;
   int        count;

   // internal building blocks: the caller never sees a node
   DoubleNode* detachFirst();
   DoubleNode* detachLast();
   DoubleNode* detachValue(int value);

public:
   DoublyLinkedList();                              // constructor
   ~DoublyLinkedList();                             // destructor
   DoublyLinkedList(const DoublyLinkedList &other);          // copy constructor
   DoublyLinkedList& operator=(const DoublyLinkedList &other);  // copy assignment

   void pushFront(int value);
   void pushBack(int value);

   bool popFront(int &outValue);      // removes AND frees, returns the value
   bool popBack(int &outValue);
   bool deleteValue(int value);

   bool contains(int value) const;
   void print() const;
   void printReverse() const;
   int  length() const;
   bool isEmpty() const;
   void clear();
};
```

DoublyLinkedList — Class Skeleton

```
class DoublyLinkedList
{
private:
    DoubleNode *headPtr;
    DoubleNode *tailPtr;
    int         count;

public:
    DoublyLinkedList();               // constructor
    ~DoublyLinkedList();              // destructor

    void pushFront(int value);
    void pushBack(int value);

    DoubleNode* detachFirst();
    DoubleNode* detachLast();
    DoubleNode* detachValue(int value);

    bool deleteFirst();
    bool deleteValue(int value);

    void print() const;
    void printReverse() const;
    int  length() const;
    bool isEmpty() const;
    void clear();
};
```

CONSTRUCTOR — EMPTY LIST

```
DoublyLinkedList::DoublyLinkedList()
    : headPtr(nullptr), tailPtr(nullptr), count(0)
{
}
```



- Runs **automatically** when the object is created.
- An empty list is `headPtr == nullptr` and `tailPtr == nullptr`.
- No caller can forget to initialize it.

```
DoublyLinkedList list;      // constructor called here
```



> Lecture 08: `Node *headPtr = NULL;` written by hand in `main`.
> Lecture 09: the **class guarantees** it.

STACK OBJECT VS HEAP OBJECT

```
void demo()
{
    DoublyLinkedList list;          // object on the STACK
    list.pushBack(10);              // nodes on the HEAP (new)
    list.pushBack(20);
}                                   // ~DoublyLinkedList() called automatically
```



- The **list object** (3 fields) lives on the stack.
- The **nodes** live on the heap, created with `new` inside the methods.
- When the function ends, the stack object dies and its destructor deletes the nodes.

NEW & DELETE

Destructor and RAII

NEW / DELETE VS MALLOC / FREE

C (lecture 08)	C++ (lecture 14)
`malloc(sizeof(Node))`	`new DoubleNode(10)`
memory only, no initialization	memory **and** constructor call
returns `NULL` on failure	throws `std::bad_alloc`
`free(ptr)`	`delete ptr`
does not know about destructors	calls the destructor, then frees

```
DoubleNode *nodePtr = new DoubleNode(10);
delete nodePtr;
nodePtr = nullptr;      // still good practice
```



> Never mix them: `new` → `delete`, `malloc` → `free`.

Destructor — Cleaning Up the Nodes

```
DoublyLinkedList::~DoublyLinkedList()
{
    clear();
}

void DoublyLinkedList::clear()
{
    DoubleNode *currentPtr = headPtr;
    while (currentPtr != nullptr)
    {
        DoubleNode *nextPtr = currentPtr->nextPtr;  // save BEFORE delete
        delete currentPtr;
        currentPtr = nextPtr;
    }
    headPtr = nullptr;
    tailPtr = nullptr;
    count   = 0;
}
```

We must save `nextPtr` **before** `delete`. After `delete`, reading `currentPtr->nextPtr` is undefined behavior.

RAII

Resource **A**cquisition **I**s **I**nitialization.

- The object acquires the resource (memory) in its constructor or methods.
- The object releases it in its destructor.
- The lifetime of the resource = the lifetime of the object.
- No `free` calls scattered through `main`.

```
C:   malloc ... free ... free ... (did I forget one?)
C++: object created → ... → object leaves scope → destructor cleans up
```

> This is the answer to "Mistake 3: memory leaks" from the C-objects lecture.

OPERATIONS

as Methods

pushFront

```
void DoublyLinkedList::pushFront(int value)
{
    DoubleNode *newPtr = new DoubleNode(value);

    newPtr->nextPtr = headPtr;
    if (headPtr != nullptr)
        headPtr->prevPtr = newPtr;
    else
        tailPtr = newPtr;          // list was empty

    headPtr = newPtr;
    ++count;
}
```

```
BEFORE:  head ──► [10] ◄──► [20] ◄── tail
AFTER :  head ──► [5] ◄──► [10] ◄──► [20] ◄── tail
```

> No `Node **headPtrPtr`. The method sees `headPtr` through `this`.

pushBack

```
void DoublyLinkedList::pushBack(int value)
{
    DoubleNode *newPtr = new DoubleNode(value);

    newPtr->prevPtr = tailPtr;
    if (tailPtr != nullptr)
        tailPtr->nextPtr = newPtr;
    else
        headPtr = newPtr;          // list was empty

    tailPtr = newPtr;
    ++count;
}
```

```
BEFORE:  head ──► [10] ◄──► [20] ◄── tail
AFTER :  head ──► [10] ◄──► [20] ◄──► [30] ◄── tail
```

> `pushFront` and `pushBack` are mirror images.
> `tailPtr` gives us O(1) insertion at the end (lecture 08 needed `_findLast`).

DETACH

Remove, but do not free

Why detach Is Private

C (lecture 08): detach is PUBLIC
Node is just a struct, there is nothing to protect.

C++ (lecture 09): detach is PRIVATE
The list owns its nodes, so nobody else
may hold a pointer to one.

If `detachFirst()` were public:

- the caller would get a raw pointer to a part of the list (broken encapsulation),
- the caller would have to remember `delete` (otherwise a leak),
- the caller could keep the pointer after `delete` (dangling pointer).

> Public API works on **values** (`int`). Nodes never leave the class.
> Every `new` and every `delete` happens **inside** the class.

detachFirst (private)

```
DoublyLinkedList::DoubleNode* DoublyLinkedList::detachFirst()
{
    if (headPtr == nullptr)
        return nullptr;

    DoubleNode *detachedPtr = headPtr;          // 1. save
    headPtr = headPtr->nextPtr;                 // 2. advance head

    if (headPtr != nullptr)
        headPtr->prevPtr = nullptr;             // new head has no predecessor
    else
        tailPtr = nullptr;                      // list became empty

    detachedPtr->nextPtr = nullptr;             // 3. isolate the node
    detachedPtr->prevPtr = nullptr;
    --count;
    return detachedPtr;
}
```

> The return type needs the full name `DoublyLinkedList::DoubleNode*`,

detachFirst

```
DoubleNode* DoublyLinkedList::detachFirst()
{
    if (headPtr == nullptr)
        return nullptr;

    DoubleNode *detachedPtr = headPtr;          // 1. save
    headPtr = headPtr->nextPtr;                 // 2. advance head

    if (headPtr != nullptr)
        headPtr->prevPtr = nullptr;             // new head has no predecessor
    else
        tailPtr = nullptr;                      // list became empty

    detachedPtr->nextPtr = nullptr;             // 3. isolate the node
    detachedPtr->prevPtr = nullptr;
    --count;
    return detachedPtr;
}
```

Same three steps as in lecture 08, plus the `prevPtr` and `tailPtr` bookkeeping.

detachLast

```
DoubleNode* DoublyLinkedList::detachLast()
{
    if (tailPtr == nullptr)
        return nullptr;

    DoubleNode *detachedPtr = tailPtr;
    tailPtr = tailPtr->prevPtr;

    if (tailPtr != nullptr)
        tailPtr->nextPtr = nullptr;
    else
        headPtr = nullptr;                  // list became empty

    detachedPtr->nextPtr = nullptr;
    detachedPtr->prevPtr = nullptr;
    --count;
    return detachedPtr;
}
```

In a singly linked list, removing the last node required walking to the **second-to-last** node: O(n).

detachValue (private)

```
DoublyLinkedList::DoubleNode* DoublyLinkedList::detachValue(int value)
{
   DoubleNode *currentPtr = headPtr;

   while (currentPtr != nullptr && currentPtr->value != value)
      currentPtr = currentPtr->nextPtr;

   if (currentPtr == nullptr) return nullptr;      // not found
   if (currentPtr == headPtr) return detachFirst();
   if (currentPtr == tailPtr) return detachLast();

   // middle node: bypass it
   currentPtr->prevPtr->nextPtr = currentPtr->nextPtr;
   currentPtr->nextPtr->prevPtr = currentPtr->prevPtr;

   currentPtr->prevPtr = nullptr;
   currentPtr->nextPtr = nullptr;
   --count;
   return currentPtr;
}
```

DETACHVALUE — THREE CASES AGAIN

```
List:  [10] ◄──► [20] ◄──► [30]

Case 1 — value at the HEAD:      → detachFirst()
Case 2 — value at the TAIL:      → detachLast()
Case 3 — value in the MIDDLE:    → bypass using prevPtr and nextPtr
Case 4 — value NOT FOUND:        → return nullptr
```

In lecture 08 we looked **one node ahead** (`currentPtr->nextPtr->value`) because we had no way back.

Now every node knows its predecessor. We stop **on** the node we want, and no lookahead is needed.

detachValue — Implementation

```
DoubleNode* DoublyLinkedList::detachValue(int value)
{
   DoubleNode *currentPtr = headPtr;

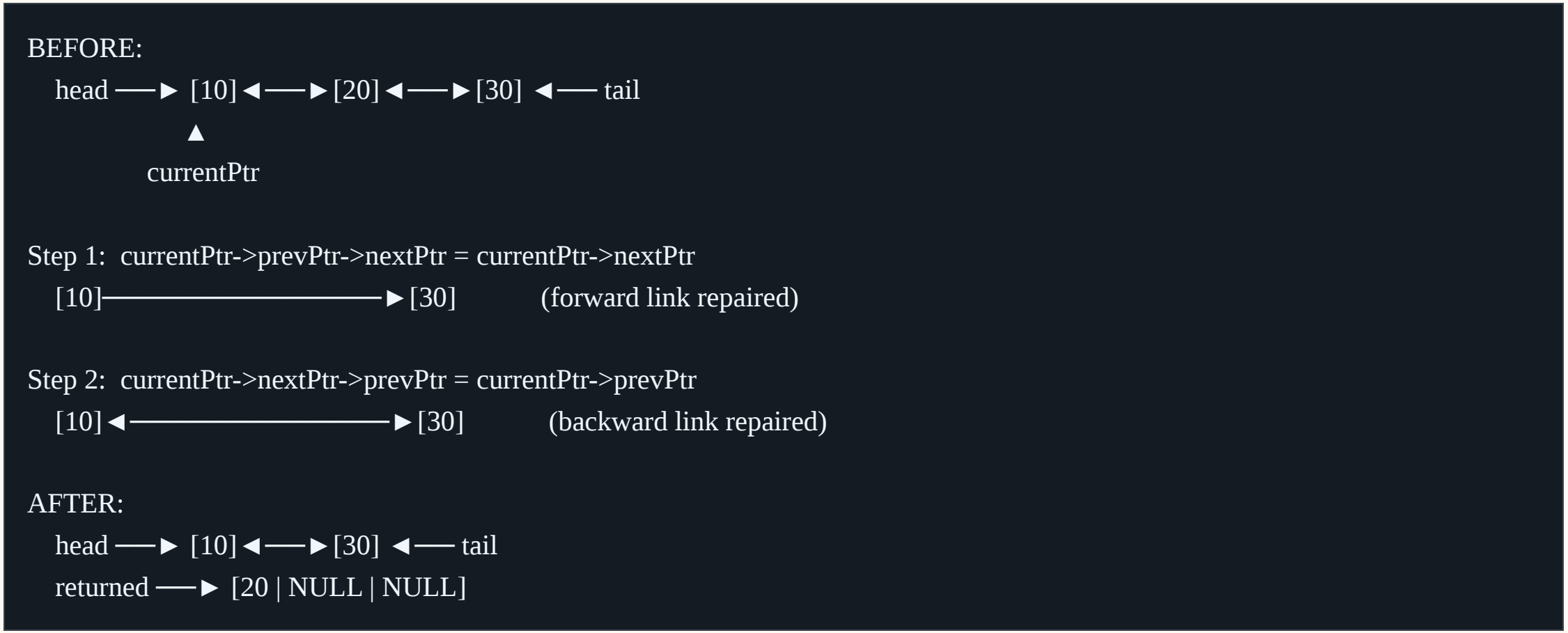
   while (currentPtr != nullptr && currentPtr->value != value)
      currentPtr = currentPtr->nextPtr;

   if (currentPtr == nullptr) return nullptr;      // not found
   if (currentPtr == headPtr) return detachFirst();
   if (currentPtr == tailPtr) return detachLast();

   // middle node — bypass it
   currentPtr->prevPtr->nextPtr = currentPtr->nextPtr;
   currentPtr->nextPtr->prevPtr = currentPtr->prevPtr;

   currentPtr->prevPtr = nullptr;
   currentPtr->nextPtr = nullptr;
   --count;
   return currentPtr;
}
```

DETACHVALUE — BYPASS IN MEMORY



> Two links to repair, not one. Forget one and `printReverse` will crash.

DELETE

detach + free

deleteFirst and deleteValue

```
bool DoublyLinkedList::deleteFirst()
{
    DoubleNode *detachedPtr = detachFirst();
    if (detachedPtr == nullptr)
        return false;

    delete detachedPtr;
    return true;
}

bool DoublyLinkedList::deleteValue(int value)
{
    DoubleNode *detachedPtr = detachValue(value);
    if (detachedPtr == nullptr)
        return false;

    delete detachedPtr;
    return true;
}
```

Same rule as in lecture 08: `delete = detach + free` .

TRAVERSAL

in both directions

print, printReverse, length, isEmpty

```cpp
void DoublyLinkedList::print() const
{
    if (headPtr == nullptr) { std::cout << "List is empty.\n"; return; }

    for (DoubleNode *p = headPtr; p != nullptr; p = p->nextPtr)
        std::cout << p->value << " ";
    std::cout << "\n";
}

void DoublyLinkedList::printReverse() const
{
    for (DoubleNode *p = tailPtr; p != nullptr; p = p->prevPtr)
        std::cout << p->value << " ";
    std::cout << "\n";
}

int  DoublyLinkedList::length()  const { return count; }
bool DoublyLinkedList::isEmpty() const { return headPtr == nullptr; }
```

`length()` is now O(1): we keep `count` up to date instead of traversing. All `const` methods promise

RULE OF THREE

Copying Is Dangerous

THE PROBLEM — SHALLOW COPY

```
DoublyLinkedList a;
a.pushBack(10);
a.pushBack(20);

DoublyLinkedList b = a;     // default copy: copies headPtr, tailPtr, count
```



The compiler-generated copy copies the **pointers**, not the nodes.

Shallow Copy — Double Delete

```
void demo()
{
    DoublyLinkedList a;
    a.pushBack(10);
    a.pushBack(20);

    DoublyLinkedList b = a;     // shallow copy
}   // ~b() deletes [10] and [20]
    // ~a() deletes [10] and [20] AGAIN  → undefined behavior
```



- ⚒ Both destructors walk the same nodes.
- ⚒ The second `delete` hits memory that is already freed (**double free**).
- ⚒ Changing `b` also changes `a`: `b.deleteFirst()` leaves `a.headPtr` dangling.

> The default copy is fine for `int`, but wrong for any class that **owns** memory.

RULE OF THREE

If a class needs **one** of these, it almost certainly needs **all three**:

1. **Destructor**: `~DoublyLinkedList()`
2. **Copy constructor**: `DoublyLinkedList(const DoublyLinkedList &other)`
3. **Copy assignment**: `DoublyLinkedList& operator=(const DoublyLinkedList &other)`

```
Owns memory (new)  →  destructor needed
                   →  so copying must be DEEP
                   →  copy constructor + copy assignment needed
```

> Our list calls `new` in `pushFront` / `pushBack`, so it owns memory.

Copy Constructor — Deep Copy

```
DoublyLinkedList::DoublyLinkedList(const DoublyLinkedList &other)
    : headPtr(nullptr), tailPtr(nullptr), count(0)
{
    for (DoubleNode *p = other.headPtr; p != nullptr; p = p->nextPtr)
        pushBack(p->value);        // brand new node for every old node
}
```



- First build an **empty** list (initializer list).
- Then walk `other` and create a **new node** for each of its nodes.
- `other` is `const`: we only read from it.
- Accessing `other.headPtr` is legal: private is per **class**, not per object.

```
DoublyLinkedList b = a;     // now b has its own nodes
```



Copy Assignment

```
DoublyLinkedList& DoublyLinkedList::operator=(const DoublyLinkedList &other)
{
   if (this != &other)            // guard against  a = a;
   {
      clear();                    // release OUR old nodes first
      for (DoubleNode *p = other.headPtr; p != nullptr; p = p->nextPtr)
         pushBack(p->value);
   }
   return *this;                  // allows  a = b = c;
}
```



Constructor vs assignment:

- Constructor: the object is **new**, so there is nothing to clean up.
- Assignment: the object **already has nodes**, so we `clear()` them first or they leak.
- Without the `this != &other` check, `a = a` would clear the list and then copy from an empty list.

CONSTRUCTOR VS ASSIGNMENT

```
DoublyLinkedList a;
a.pushBack(10);

DoublyLinkedList b = a;     // copy CONSTRUCTOR (b is being created)
DoublyLinkedList c;
c = a;                      // copy ASSIGNMENT  (c already exists)
```



	Copy constructor	Copy assignment
Object exists before?	no	yes
Must free old nodes?	no	yes, `clear()`
Self-assignment check?	no	yes
Returns	nothing	`*this`

DISABLING COPY

Sometimes copying a list makes no sense, or is too expensive. Then we **forbid** it:

```
DoublyLinkedList(const DoublyLinkedList &) = delete;
DoublyLinkedList& operator=(const DoublyLinkedList &) = delete;
```



```
DoublyLinkedList b = a;     // compile error — caught at build time
```



A compile error is better than a runtime crash.

Modern C++ adds a **Rule of Five** (move constructor and move assignment). We leave that for a later lecture.

PUTTING IT

Together

main — Full Demo

```
int main()
{
    DoublyLinkedList list;
    list.pushBack(10);
    list.pushBack(20);
    list.pushBack(30);
    list.pushFront(5);

    list.print();                // 5 10 20 30
    list.printReverse();         // 30 20 10 5

    DoublyLinkedList copy = list;   // deep copy
    copy.deleteValue(20);
    copy.print();                // 5 10 30
    list.print();                // 5 10 20 30  (unchanged)

    DoubleNode *nodePtr = list.detachFirst();   // we own it now
    std::cout << nodePtr->getValue() << "\n";   // 5
    delete nodePtr;

    std::cout << list.length() << "\n";         // 3
    return 0;
}   // ~copy(), ~list() → all remaining nodes freed automatically
```

MEMORY CHECKLIST

```
Every  new       →  exactly one  delete
Every  detach    →  caller must  delete  (or reinsert)
Destructor       →  frees all nodes still inside the list
Copy             →  deep, never shallow
Assignment       →  clear() old nodes before copying
```

How to check for leaks:

```
valgrind --leak-check=full ./a.out
g++ -fsanitize=address -g main.cpp && ./a.out
```

> All heap blocks were freed -- no leaks are possible is the goal.

SUMMARY

Lecture 08 (C)	Lecture 14 (C++)	Memory
`createNode`	`new DoubleNode(v)` inside `push*`	heap
`destroyNode`	`delete` inside destructor / `delete*`	heap
`Node **headPtrPtr`	member `headPtr`	n/a
`detachFirst(&head)`	`list.detachFirst()`	caller owns result
`deleteFirst(&head)`	`list.deleteFirst()`	freed
manual cleanup	`~DoublyLinkedList()`	RAII

> Composition: the list **owns** its nodes, so the list must clean them up.

ONE RULE TO REMEMBER

```
detach  →  structure operation  →  caller owns the node
delete  →  memory operation     →  node is freed

new     in methods  +  delete in destructor  =  RAII
destructor needed   →  copy constructor + copy assignment needed
```

> If you understand composition, you understand who calls `delete`.
> If you understand RAII, you never write `free` in `main` again.

COMMON MISTAKES

```
// 1. Reading a node after delete
delete currentPtr;
currentPtr = currentPtr->nextPtr;     // WRONG — save nextPtr first

// 2. Forgetting one of the two links
currentPtr->prevPtr->nextPtr = currentPtr->nextPtr;
// missing: currentPtr->nextPtr->prevPtr = currentPtr->prevPtr;

// 3. Forgetting tailPtr when the list becomes empty
headPtr = headPtr->nextPtr;          // tailPtr still points to the old node

// 4. Mixing allocators
DoubleNode *p = new DoubleNode(1);
free(p);                             // WRONG — new → delete
```

THANK

You

PUBLIC API

pop and delete: detach + free

popFront, popBack, deleteValue

```cpp
bool DoublyLinkedList::popFront(int &outValue)
{
   DoubleNode *nodePtr = detachFirst();    // structure operation (private)
   if (nodePtr == nullptr)
      return false;

   outValue = nodePtr->value;              // copy the data OUT
   delete nodePtr;                         // memory operation: list frees its own node
   return true;
}

bool DoublyLinkedList::popBack(int &outValue)
{
   DoubleNode *nodePtr = detachLast();
   if (nodePtr == nullptr)
      return false;

   outValue = nodePtr->value;
   delete nodePtr;
   return true;
}

bool DoublyLinkedList::deleteValue(int value)
{
   DoubleNode *nodePtr = detachValue(value);
   if (nodePtr == nullptr)
      return false;

   delete nodePtr;
   return true;
}
```

RULE STAYS THE SAME

```
detach  →  structure operation              →  PRIVATE, node still exists
pop     →  detach + copy value out + delete  →  PUBLIC
delete  →  detach + delete                   →  PUBLIC

public methods:  values go out, nodes never do
```

Lecture 08: `delete = detach + destroyNode`.

Lecture 09: the same formula, but both halves are now **hidden inside the class**.

> The caller never writes `delete`. The class guarantees it.

const Methods

```
bool DoublyLinkedList::contains(int value) const
{
    for (DoubleNode *p = headPtr; p != nullptr; p = p->nextPtr)
        if (p->value == value)
            return true;
    return false;
}

void show(const DoublyLinkedList &list)    // const reference, no copy
{
    list.print();          // OK:    print() is const
    list.contains(5);      // OK:    contains() is const
    list.pushBack(1);      // ERROR: pushBack() is not const
}
```



`const` **after the parentheses** means: "this method does not modify `*this`".

It has nothing to do with the return type (`void`, `bool`, `int`...).

CONST METHODS: NOTES

```
const int* f();        const BEFORE the name  →  about the RETURN TYPE
void print() const;    const AFTER the ()     →  about the OBJECT (this)
```

- Inside a `const` method, `this` is `const DoublyLinkedList*`.
- A `const` method can call only other `const` methods.
- A `const` object or `const &` can call only `const` methods.
- Constness is **shallow**: `headPtr` itself is protected, but the node it points to is not.

> The compiler will not stop `headPtr->value = 999;` inside a `const` method.
> That part is still the programmer's discipline.

Copy Constructor: Deep Copy

```
DoublyLinkedList::DoublyLinkedList(const DoublyLinkedList &other)
    : headPtr(nullptr), tailPtr(nullptr), count(0)
{
    for (DoubleNode *p = other.headPtr; p != nullptr; p = p->nextPtr)
        pushBack(p->value);        // brand new node for every old node
}
```



- First build an **empty** list (initializer list).
- Then walk `other` and create a **new node** for each of its nodes.
- `other` is `const`: we only read from it.
- `other.headPtr` is legal: `private` is per **class**, not per object.

```
DoublyLinkedList b = a;   // copy constructor: deep copy (value semantics)
```



VALUE SEMANTICS: C++ VS JAVA / PYTHON

```
Java / Python:   b = a       →  both names point to ONE object (shallow)
                      b = a.clone() for an independent copy

C++ (value):     b = a       →  b is an independent object (deep),
                      because the class defines its copy

Compiler default copy  =  shallow  =  BUG for a class that owns memory
```

In C++ the class author decides what `=` means.

`std::vector`, `std::string` and `int` all copy **by value**.

Our list must behave the same way.

> A shallow copy of two owners of the same nodes means two `delete`s.
> Sharing ownership is a different design (`std::shared_ptr`).

Alternative: Forbid Copy, Offer an Explicit Copy

```
// in the class (INSTEAD of copy constructor and copy assignment):
DoublyLinkedList(const DoublyLinkedList &) = delete;
DoublyLinkedList& operator=(const DoublyLinkedList &) = delete;

void cloneInto(DoublyLinkedList &target) const;
```



```
void DoublyLinkedList::cloneInto(DoublyLinkedList &target) const
{
    if (this == &target) return;
    target.clear();
    for (DoubleNode *p = headPtr; p != nullptr; p = p->nextPtr)
        target.pushBack(p->value);
}
```



```
DoublyLinkedList b = a;     // compile error: caught at build time
DoublyLinkedList b;
a.cloneInto(b);             // explicit: "I know this costs O(n)"
```



A `clone()` that returns the list **by value** would need a move constructor (Rule of Five), so we use

main: Full Demo

```
int main()
{
    DoublyLinkedList list;
    list.pushBack(10);
    list.pushBack(20);
    list.pushBack(30);
    list.pushFront(5);

    list.print();                // 5 10 20 30
    list.printReverse();         // 30 20 10 5

    DoublyLinkedList copy = list;   // copy constructor: deep copy (value semantics)
    copy.deleteValue(20);
    copy.print();                // 5 10 30
    list.print();                // 5 10 20 30  (unchanged)

    int v;
    if (list.popFront(v))        // the list frees the node itself
        std::cout << v << "\n";     // 5

    std::cout << list.length() << "\n";     // 3
    std::cout << std::boolalpha << list.contains(20) << "\n";   // true
    return 0;
}   // ~copy(), ~list() → all remaining nodes freed automatically
```

There is no `delete` in `main`. There is not even a node in `main`.

MEMORY CHECKLIST

```
Every  new       →  exactly one  delete, both INSIDE the class
pop / delete     →  the list frees the node itself
Destructor       →  frees all nodes still inside the list
Copy             →  deep, or forbidden. Never shallow.
Assignment       →  clear() old nodes before copying
```

How to check for leaks:

```
valgrind --leak-check=full ./a.out
g++ -fsanitize=address -g main.cpp && ./a.out
```

> All heap blocks were freed -- no leaks are possible is the goal.

SUMMARY

Lecture 08 (C)	Lecture 14 (C++)	Who frees?
`createNode`	`new DoubleNode(v)` inside `push*`	n/a
`destroyNode`	`delete` inside the class	the list
`Node **headPtrPtr`	member `headPtr`	n/a
`detachFirst(&head)` public	`detachFirst()` **private**	n/a
`deleteFirst(&head)`	`list.popFront(v)`	the list
manual cleanup	`~DoublyLinkedList()`	RAII

> Composition: the list **owns** its nodes, so the list must clean them up.

ONE RULE TO REMEMBER

```
detach  →  private structure operation
pop /
delete  →  public operation: detach + delete

new in methods  +  delete in destructor  =  RAII
destructor needed  →  copy constructor + copy assignment
                      (or forbid copying with = delete)
```

> If you understand composition, you understand who calls `delete`.
> If you understand RAII, you never write `free` in `main` again.

EXERCISES

1. Draw the **UML class diagram** for `Stack` built on top of `DoublyLinkedList`. Is it **composition** or **inheritance**? Justify your choice.
2. Add `bool insertAfter(int target, int value)`. Handle: empty list, `target` at the tail (update `tailPtr`), `target` not found.
3. Add `void reverse()` that reverses the list **in place**: swap `prevPtr` and `nextPtr` in every node, then swap `headPtr` and `tailPtr`. No `new` allowed.
4. Add `bool removeAt(int index)`. Walk from `headPtr` or from `tailPtr`, whichever is closer. Reuse the private `detach` logic and free the node **inside** the class.

> Rule for all exercises: the public API takes and returns **values**.
> No method may return or accept a `DoubleNode*`.

EXERCISES: MEMORY LAB

1. Comment out the body of `~DoublyLinkedList()`. Run `valgrind --leak-check=full`. How many bytes leak, and why?
2. Remove the copy constructor and run `DoublyLinkedList b = a;` with `-fsanitize=address`. Which error do you get?
3. Remove the `this != &other` check and run `a = a;`. What happens to the list?
4. Temporarily make `detachFirst()` **public**, call it from `main` and do not free the result. Does the destructor free that node? Explain using composition, and explain why this is a bad design.

> Break the code on purpose, read the error message, then fix it.
> This is how you learn what each line is protecting you from.

DESIGN PRINCIPLES

What guided our decisions

WHAT IS SOLID?

SOLID is a set of five object-oriented design guidelines (popularized by Robert C. Martin). They are not laws of nature. They are **rules of thumb** that help keep code easy to change, test and not break.

Letter	Principle	One-line meaning
S	Single Responsibility	one class, one reason to change
O	Open / Closed	extend behavior without editing working code
L	Liskov Substitution	a derived object must work wherever the base is expected
I	Interface Segregation	do not force a class to have methods it cannot honor
D	Dependency Inversion	depend on abstractions, not on concrete details

> Today we met **S, L, I** in real code.
>
> **O** and **D** need interfaces and polymorphism, so they come later.

S: SINGLE RESPONSIBILITY

A class should have one reason to change.

How we applied it:

- `DoubleNode` knows only `value`, `prevPtr`, `nextPtr`. It does not print, sum or search.
- `DoublyLinkedList` is responsible for **memory and navigation**: `new`, `delete`, links.
- Processing the data (sum, filter, format) belongs to the **client code**.

> Honest note: our list has `print()`. That is a teaching convenience.
> A production list would expose iteration, and the client would decide what to do with each value.

L: Liskov Substitution

```c
// A tempting but WRONG design:
struct Node       { int value; Node *nextPtr; };
struct DoubleNode : Node { Node *prevPtr; };     // "a doubly node IS a node"

// Old code written for Node:
void insertAfter(Node *posPtr, Node *newPtr)
{
   newPtr->nextPtr = posPtr->nextPtr;
   posPtr->nextPtr = newPtr;
   // prevPtr is never updated!
}
```

Pass a `DoubleNode*` to this function and the forward chain is correct, but the `prevPtr` links are **silently corrupted**. `printReverse()` will crash later, far from the real bug.

A derived object must keep the **contract and invariants** of the base. `DoubleNode` cannot, so it must **not** inherit from `Node`.

> "Is-a" in English is not always "is-a" in code.

I: INTERFACE SEGREGATION

Do not force a class to implement methods it cannot honor.

Rejected idea: put `getPrev()` in a common list interface.

- For a doubly linked list it returns a real node.
- For a singly linked list it would return `nullptr`, which is a **lie**: the question has no valid answer there.
- Users would have to guess which lists "really" support it.

```
Singly list  →  only forward operations
Doubly list  →  forward + backward operations
```

> Smaller, honest interfaces beat one big interface with fake methods.

ZERO-OVERHEAD PRINCIPLE

What you don't use, you don't pay for. (Bjarne Stroustrup, the core idea of C++)

We rejected `virtual` in the node. A class with a virtual function stores a hidden **`vptr`** in every object.

```
64-bit system, typical layout:

DoubleNode (no virtual):   int 4 + padding 4 + 2 pointers 16  =  24 bytes
DoubleNode (with virtual): + vptr 8                           =  32 bytes

+33% memory per node. A list of a million nodes: +8 MB for nothing.
```

Plus: virtual calls are indirect jumps, which are harder for the CPU to predict and block inlining. Our node has no polymorphic behavior, so it has no reason to pay.

TYPE SAFETY

Let the compiler know what is in memory.

C solution (generic container): `void* data`. The compiler knows nothing, and you can read garbage:

```c
void *dataPtr = &someInt;
double d = *(double*)dataPtr;       // compiles. Reads garbage.
```



```c
struct DoubleNode { int value; /* ... */ };
int v = nodePtr->value;             // the type is known, nothing to cast
```



Also `nullptr` instead of `NULL`, `const` methods, and a private node type all move errors from **runtime** to **compile time**.

Next step for other value types: templates or `std::variant`.

DRY IN CONTEXT

DRY = Don't Repeat Yourself. Naive reading: "never write two similar lines."

```c
struct Node       { int value; Node *nextPtr; };
struct DoubleNode { int value; DoubleNode *prevPtr; DoubleNode *nextPtr; };
//                  ^^^^^^^^ repeated on purpose
```



We could remove the repetition with inheritance, but that creates a **wrong is-a relation** (see the LSP slide).

DRY is about duplicated **knowledge and logic**, not about duplicated lines. Two structs that happen to share a field may evolve differently.

> One repeated line is cheaper than a false inheritance relation.

SUMMARY: PRINCIPLE → OUR DECISION

Principle	Our decision in the code
S Single Responsibility	node holds data and links only; the list owns memory and navigation
L Liskov Substitution	`DoubleNode` does **not** inherit from `Node`
I Interface Segregation	no fake `getPrev()` on a singly linked list
Zero-Overhead	no `virtual` in the node: 24 bytes, not 32
Type Safety	typed `int value`, no `void*`, `nullptr`, `const` methods
DRY in context	a repeated field is better than a wrong inheritance

WHAT THESE PRINCIPLES DO (AND DON'T DO)

They help prevent:

- a corrupted structure (`prevPtr` silently out of sync)
- hidden costs (an unneeded `vptr` in every node)
- unclear contracts (methods that cannot honestly work)
- unsafe reads of memory (`void*` casts)

They do not replace:

- **RAII** and the Rule of Three, which are what actually prevent leaks
- testing and tools such as Valgrind or AddressSanitizer

> SOLID is not abstract theory from a slide.
>
> It is a set of tools we used to decide **what to write and what to refuse to write**.

FINAL QUESTION

Before you add a method, a base class or a `virtual`, ask:

1. Does this class have **one** reason to change?
2. Can a derived object **really** replace the base everywhere?
3. Am I forcing someone to implement something they **cannot honor**?
4. Am I paying (memory, speed) for something I **don't use**?